Kick Start 2020 - Round G

Analysis: Combination Lock

We can see that if we decide a final value at which all wheels should be in the end, moves for each wheel to reach that value are independent of the moves performed on other wheels. Thus, we can calculate number of moves for each wheel separately. Consider a wheel i which is at value x currently. We want the wheel to finally reach the value y. There are 2 cases here:

- Case 1: $x \leq y$. Increasing the value of the wheel i would take y-x steps. Decreasing the value of wheel i would take **N** - y + x steps. Hence, the minimum number of moves in this case would be minimum of y-x and **N** - y + x.
- Case 2: $x > y$. Decreasing the value of the wheel i would take x-y steps. Increasing the value of wheel i would take **N** + y - x steps. Hence, the minimum number of moves in this case would be minimum of x-y and **N** + y - x.

Test Set 1

We can solve this test set by trying all possible **N** values that all the wheels should have in the end. For each value, we calculate the total number of moves to get all of the wheels to this value using Case 1 and Case 2. The answer is the minimum possible moves performed over all such X. There are **N** possible values and for each value, we perform O(**W**) operations. Thus, the time complexity of the solution would be O(**W** × **N**).

Test Set 2

A major observation is that we could always get the minimum possible moves by finally bringing all the wheels to one of the initial values of the given wheels. We can prove this by considering a value Val which is not among the initial values of wheels and then showing that we can get a same or better answer by moving all the wheels to one of the initial values. Consider the initial values of the wheels in the sorted order. Let the index of the wheel with smallest initial value greater than Val be j, if no such value exists, then we can consider j = 1 as it is first wheel next to value Val in cyclic order. Let the index of wheel with largest initial value smaller than Val be i, if no such value exists, then we can consider i = **W** as it is the first wheel before value Val in cyclic order. Now to reach the value Val, each value will either reach $\mathbf{X_i}$ or $\mathbf{X_j}$. Now say there are y wheels at value $\mathbf{X_i}$ currently which leaves us with **W** - y wheels at value $\mathbf{X_j}$. The number of moves for all wheels to reach value Val would be $y \times (Val - \mathbf{X_i}) + (\mathbf{W} - y) \times (\mathbf{X_j} - Val)$. We can get the same or better result than this. There are 2 possibilities:

- $y \leq \mathbf{W} - y$. If we choose to bring all the wheels to value $\mathbf{X_j}$, we will have number of moves as $y \times (Val - \mathbf{X_i}) + y \times (\mathbf{X_j} - Val)$, which is never large than the number of moves required for all wheels to reach value Val. Hence, we have a better answer if we bring all the wheels to value $\mathbf{X_j}$.
- $y \geq \mathbf{W} - y$. If we choose to bring all the wheels to value $\mathbf{X_i}$, we will have number of moves as $(\mathbf{W} - y) \times (Val - \mathbf{X_i}) + (\mathbf{W} - y) \times (\mathbf{X_j} - Val)$, which is never large than the number of moves required for all wheels to reach value Val. Hence, we have a better answer if we bring all the wheels to value $\mathbf{X_i}$.

Now instead of trying all possible values from 1 to **N**, we would try the initial values of the wheels and calculate moves required for all wheels to reach that value. For each value, we take O(**W**) time to calculate the moves required for all wheels to reach that value. There are **W** values. Hence the complexity is $O(\mathbf{W}^2)$.

Test Set 3

We have already proved that we only need to consider one of the initial values of the wheels. We need to calculate the number of moves required for all wheels to reach a particular value efficiently. We can do the following. Sort the initial values of the wheels. Maintain a prefix sum array Pre. Pre[i] denotes the sum of initial values of wheels from 1 to i. We define a method GetSum(i,j) which gives the sum of values between indexes i and j. This can be calulated in O(1) using Pre array.

Suppose that currently we are calulating the number of moves required for all wheels to reach $\mathbf{X_i}$. Consider any wheel j from index 1 to i. Minimum number of moves required for wheel j to reach value $\mathbf{X_i}$ would be minimum of $\mathbf{X_i} - \mathbf{X_j}$ and $\mathbf{N} - \mathbf{X_i} + \mathbf{X_j}$. Consider 2 indexes k and l such that $1 \le k < l \le i$. We can prove that it is not possible to have $\mathbf{X_i} - \mathbf{X_k} < \mathbf{N} - \mathbf{X_i} + \mathbf{X_k}$ and $\mathbf{N} - \mathbf{X_i} + \mathbf{X_l} < \mathbf{X_i} - \mathbf{X_l}$ simultaneously. This is because if we add the two inequalities, we get $\mathbf{N} - \mathbf{X_k} + \mathbf{X_l} < \mathbf{N} + \mathbf{X_k} - \mathbf{X_l}$ which implies $\mathbf{X_l} < \mathbf{X_k}$. But it is not possible to have such condition because we know that $\mathbf{X_k} \le \mathbf{X_l}$. Thus, we can say that there exists an index p such that $1 \le p \le i$ and for each wheel q such that $p \le q \le i$ will have minimum number of moves as $\mathbf{X_i} - \mathbf{X_q}$ and for each wheel r such that $1 \le r \le p-1$ will have minimum number of moves as $\mathbf{N} - \mathbf{X_i} + \mathbf{X_r}$. We can find this index p using binary search on wheels 1 to i. Now we need to find sum of $\mathbf{X_i} - \mathbf{X_q}$ for each q. This can be calculated in O(1) by $(i - p + 1) \times \mathbf{X_i}$ - GetSum(p,i). Now we need to find sum of $\mathbf{N} - \mathbf{X_i} + \mathbf{X_r}$ for each r. This can be calculated in O(1) by $(p-1) \times (\mathbf{N} - \mathbf{X_i})$ + GetSum(1,p-1). We have calculated number of moves required for wheels 1 to i to reach value $\mathbf{X_i}$.

Similarly, we can say that there exists an index b such that $i \le b \le \mathbf{W}$ and for each wheel c such that $i \le c \le b$ will have the minimum number of moves as $\mathbf{X_c} - \mathbf{X_i}$ and for each wheel d such that $b+1 \le d \le \mathbf{W}$ will have the minimum number of moves as $\mathbf{N} - \mathbf{X_d} + \mathbf{X_i}$. We can find this index b by using binary search on wheels 1 to **W**. Now we need to find sum of $\mathbf{X_c} - \mathbf{X_i}$ for each c. This can be calculated in O(1) by GetSum(i+1,c) - $(c-i) \times \mathbf{X_i}$. Now we need to find sum of $\mathbf{N} - \mathbf{X_d} + \mathbf{X_i}$ for each d. This can be calculated in O(1) by $(\mathbf{W} - b) \times (\mathbf{N} + \mathbf{X_i})$ - GetSum(b+1,**W**).

We have calculated the number of moves required for all wheels to reach a particular value in O(log **W**). Now we need to take the minimum over all such values. Thus we can calculate moves for all the initial values of all wheels in O($\mathbf{W} \times \log \mathbf{W}$) time complexity. Note that instead of using binary search to find indexes p and b, we can use two pointers approach to find them. We can prove that indexes p and b will keep on increasing as we increment i. The overall complexity of the solution remains same due to the sorting part.

Kick Start 2020 - Round G

Analysis: Kick_Start

The given problem statement can be rephrased as counting the number of substrings (fragments) which begin with string 'KICK' and end with string 'START'. Another thing to note is that a substring starting with 'KICK' can form multiple lucky fragments ending at different 'START'. For example, consider the string 'KICKSTARTSTART'. This string contains two lucky fragments, first being 'KICKSTART' and second being 'KICKSTARTSTART'.

Test Set 1

For a substring to be considered as a lucky fragment, we can check if it starts with 'KICK' and ends with 'START'. For every substring from i-th to j-th index, where $0 \leq i < j <$ **N**, we can check if the substring begins with 'KICK' by comparing whether i-th index is equal to 'K', (i+1)-th index is equal to 'I', ... , (i+3)-th index is equal to 'K'. Similarly we can check if the substring ends with 'START' by comparing whether (j-4)-th index is equal to 'S', (j-3)-th index is equal to 'T', ... , j-th index is equal to 'T'. Note that we only check the characters if the indices are within the bounds of the substring. Hence, it takes constant number of operations to check whether a substring is lucky fragment or not. In total, we have $O(\mathbf{N}^2)$ substrings to check. Therefore, total time complexity for the approach is $O(\mathbf{N}^2)$.

Another solution could be to use two loops iterating over the string. Outer loop is used to scan the string for 'KICK'. Whenever 'KICK' is encountered, we use the inner loop to scan the remaining string i.e. substring to the right of 'KICK' to count the occurrences of string 'START'. We repeat this for every 'KICK' encountered using the outer loop and keep summing up the count of occurrences of string 'START' to get the total number of lucky fragments. Rationale is that every 'START' encountered can be paired up with the 'KICK' found using outer loop to create a lucky fragment for Ksenia.
As we iterate over the string using two loops, the overall time complexity for this solution is $O(\mathbf{N}^2)$.

Test Set 2

We take a single pass over the string. During this pass, we perform following two checks:

- If we encounter string 'KICK', we increase a counter, let it be X.
- If we encounter string 'START', we add the number of 'KICK' encountered till now to the final answer, in other words add the current value of X to the final answer.

Rationale is that every 'START' we encounter can be paired up with any 'KICK' encountered before to create a lucky fragment.
Time complexity is $O(\mathbf{N})$ for the solution as we take a single pass over the string.

Kick Start 2020 - Round G

Analysis: Maximum Coins

Terminology: In what follows, when we refer to a *diagonal* starting from (x, y), we mean all cells (p, q) such that x - y = p - q. Only cells satisfying these conditions are considered, because Mike can only move diagonally.

Test Set 1

Mike is allowed to go from cell (i,j) to cell (i+1,j+1) or to cell (i-1,j-1). We can consider each possible starting point and try calculating the value of each possible path Mike can traverse. Initialize the answer as 0. For each starting cell (i,j), Mike can either go diagonally upwards or diagonally downwards. First consider all the paths which start at cell (i,j) and end diagonally above it. We keep on adding the value of coins by traversing upwards diagonally and updating the answer. Similarly, we consider all paths which start at cell (i,j) and end diagonally below it and update the answer. Each path can contain at most O(**N**) elements. Thus, it takes O(**N**) time for each starting position. There can be O($\mathbf{N}^2$) starting positions. Thus, the overall complexity of the solution is O($\mathbf{N}^3$).

Sample Code(C++)

```
int GetMaximumCoins(const vector< vector< int > >& coins) {
  int answer = 0;
  for(int i = 0; i < coins.size(); i++) {
    for(int j = 0; j < coins[0].size(); j++) {
    int currx = i, curry = j, currval = 0;
    // Go above.
    while(currx >= 0 and curry >= 0) {
      currval += coins[currx][curry];
      answer = max(answer, currval);
      currx--;
      curry--;
    }
    currx = i;
    curry = j;
    currval = 0;
    // Go below.
    while(currx < coins.size() and curry < coins[0].size()) {
      currval += coins[currx][curry];
      answer = max(answer, currval);
      currx++;
      curry++;
    }
   }
  }
  return answer;
}
```

Test Set 2

An important observation here is that all the numbers are positive. Thus, it is always optimal to collect the coins for each cell present on a particular diagonal instead of choosing some of the cells on the diagonal. This can be done by starting on the top left of each diagonal and traversing down and adding the coins collected. For each diagonal it would take O(**N**) time to calculate the coins present in that diagonal. There are O(**N**) diagonals present in the matrix. Thus, the overall time complexity of the solution is O($\mathbf{N}^2$).

Sample Code(C++)

```
long long int GetDiagonalSum(const vector< vector< int > >& coins, int i, int j) {
   long long int currval = 0;
   int currx = i, curry = j;
   while(currx < coins.size() 0 && curry < coins[0].size()) {
     currval += coins[currx][curry];
     currx++;
     curry++;
   }
  return currval;
}

long long int GetMaximumCoins(const vector< vector< int > >& coins) {
  long long int answer = 0;
  // Top row.
  for(int i = 0; i < coins[0].size(); i++)
    answer = max(answer, GetDiagonalSum(coins, 0, i));
  // Left column.
  for(int i = 0; i < coins.size(); i++)
    answer = max(answer, GetDiagonalSum(coins, i, 0));
  return answer;
}
```

Kick Start 2020 - Round G

Analysis: Merge Cards

Test Set 1

In the i-th round, Panko will have **N** - i choices. In total, there are (**N** - 1)! possibilities. Using an exhaustive approach, the expected value can be computed by definition.

Test Set 2

After each round, the number on each card equals to the sum of a subarray of **A**. In the last round, there are **N** - 1 cases:

- $\mathbf{A_1}$, $\mathbf{A_2} + \mathbf{A_3} + ... + \mathbf{A_N}$
- $\mathbf{A_1} + \mathbf{A_2}$, $\mathbf{A_3} + \mathbf{A_4} + ... + \mathbf{A_N}$

  ⋮
- $\mathbf{A_1} + \mathbf{A_2} + ... + \mathbf{A_i}$, $\mathbf{A_{i+1}} + \mathbf{A_{i+2}} + ... + \mathbf{A_N}$

  ⋮
- $\mathbf{A_1} + \mathbf{A_2} + ... + \mathbf{A_{N-1}}$, $\mathbf{A_N}$

The last round contributes a fixed number, $\mathbf{A_1} + \mathbf{A_2} + ... + \mathbf{A_N}$, to the answer. But in each case, the part contributed by the previous rounds is actually the sum of the answers of two variants of the original problem:

- Replace **A** by $\mathbf{A_1}$, $\mathbf{A_2}$, ..., $\mathbf{A_i}$
- Replace **A** by $\mathbf{A_{i+1}}$, $\mathbf{A_2}$, ..., $\mathbf{A_N}$

This part is then a weighted average of those **N** - 1 sums. However, each case is equally likely to occur. To reach a specific case in the last round, Panko has to avoid exactly one choice in each of the previous rounds. Thus arithmetic mean can be used here.

There are $O(\mathbf{N}^2)$ different subproblems. The answer of each of subproblem can be computed in O(**N**) by dynamic programming . The overall time complexity is then $O(\mathbf{N}^3)$.

Test Set 3

The above solution can be optimized to achieve $O(\mathbf{N}^2)$ time complexity by maintaining the prefix sums and suffix sums of the answers to the subproblems. But there is a faster approach that the $O(\mathbf{N}^2)$ part is in precomputation instead of having $O(\mathbf{N}^2)$ per test.

Let $\text{solve}_{\mathbf{N}}(\mathbf{A_1}, \mathbf{A_2}, ..., \mathbf{A_N})$ be a function that takes the **N** numbers written on the cards as parameters and returns the expected total score. It can be expressed as the average of **N** - 1 numbers. Each corresponds to a different choice in the first round. In particular, they are:

- $\mathbf{A_1} + \mathbf{A_2} + \text{solve}_{\mathbf{N}-1}(\mathbf{A_1} + \mathbf{A_2}, \mathbf{A_3}, ..., \mathbf{A_i}, \mathbf{A_{i+1}}, ..., \mathbf{A_{N-1}}, \mathbf{A_N})$
- $\mathbf{A_2} + \mathbf{A_3} + \text{solve}_{\mathbf{N}-1}(\mathbf{A_1}, \mathbf{A_2} + \mathbf{A_3}, ..., \mathbf{A_i}, \mathbf{A_{i+1}}, ..., \mathbf{A_{N-1}}, \mathbf{A_N})$

  ⋮
- $\mathbf{A_i} + \mathbf{A_{i+1}} + \text{solve}_{\mathbf{N}-1}(\mathbf{A_1}, \mathbf{A_2}, \mathbf{A_3}, ..., \mathbf{A_i} + \mathbf{A_{i+1}}, ..., \mathbf{A_{N-1}}, \mathbf{A_N})$

  ⋮
- $\mathbf{A_{N-1}} + \mathbf{A_N} + \text{solve}_{\mathbf{N}-1}(\mathbf{A_1}, \mathbf{A_2}, \mathbf{A_3}, ..., \mathbf{A_i}, \mathbf{A_{i+1}}, ..., \mathbf{A_{N-1}} + \mathbf{A_N})$

By using mathematical induction with $\text{solve}_2(\mathbf{A_1}, \mathbf{A_2}) = \mathbf{A_1} + \mathbf{A_2}$ as the base case, it can be shown that $\text{solve}_{\mathbf{N}}(\mathbf{A_1}, \mathbf{A_2}, ..., \mathbf{A_N})$ is a linear combination of $\mathbf{A_1}, \mathbf{A_2}, ..., \mathbf{A_N}$, which means $\text{solve}_{\mathbf{N}}(\mathbf{A_1}, \mathbf{A_2}, ..., \mathbf{A_N}) = k_{\mathbf{N}, 1} \times \mathbf{A_1} + k_{\mathbf{N}, 2} \times \mathbf{A_2} + ... + k_{\mathbf{N}, \mathbf{N}} \times \mathbf{A_N}$ where $k_{i, j}$ are constants. If all $k_{i, j}$ are precomputed, the answer for each test can be computed in time complexity $O(\mathbf{N})$.

Starting from $k_{2, 1} = k_{2, 2} = 1$, $k_{i, j}$ can be computed in increasing order of i. The transition formulas can be obtained by transforming the formula of $\text{solve}_{\mathbf{N}}(\mathbf{A_1}, \mathbf{A_2}, ..., \mathbf{A_N})$. For example, the transition formula of $k_{\mathbf{N}, 1}$ can be obtained by replacing $\text{solve}_{\mathbf{N} - 1}(...)$ and $\text{solve}_{\mathbf{N}}(...)$ by the expressions with $k_{i, j}$, then comparing the coefficients of the $\mathbf{A_1}$ term.

The j-th card will become either (part of) the (j - 1)-th card or (part of) the j-th card after a round. Correspondingly, only the coefficients of the $k_{i - 1, j - 1}$ term, the $k_{i - 1, j}$ term and the constant term can be non-zero in the transition formula of $k_{i, j}$. By grouping the like terms, the number of terms in each transition formula is reduced to at most 3. Then the overall time complexity becomes $O(\mathbf{N}^2)$.

The error analysis is straightforward since $\mathbf{A_i}$ cannot be negative. It should not be a problem in most of the implementations.